

PPAW - Laborator 12

Objective

1. Cache în memorie (MemoryCache)

1. Cache în memorie (MemoryCache)

Exercițiu : Implementați în cadrul nivelului Servicii (sau BusinessLayer) mecanismul de cache pentru entitățile principale cu care lucrați [1]. Resetați acest cache când faceți actualizări asupra entităților (add, update, delete).

Etape 1: Implementați metodele minime necesare pentru menținerea unui cache într-o clasă specifică. Exemplu C# utilizând implementarea implicită System.Runtime.Caching.MemoryCache:

```
public class MemoryCacheService : ICache
{
    private readonly int _cacheTime = 60;

    /// <param name="cacheTime">The life time in minutes.</param>
    public MemoryCacheService(int cacheTime = 60)
    {
        _cacheTime = cacheTime;
    }

    protected ObjectCache Cache
    {
        get { return MemoryCache.Default;
            // o colectie de KeyValuePair<string,object>
        }
    }

    /// Gets the value associated with the specified key.
    public T Get<T>(string key)
    {
        BinaryFormatter deserializer = new BinaryFormatter();
        using (MemoryStream memStream = new MemoryStream((byte[]) Cache[key]))
        {
            return (T)deserializer.Deserialize(memStream);
        }
    }

    /// Adds the specified key and object to the cache.
    public void Set(string key, object objectData, int? cacheTime = null)
    {
        if (objectData == null)
        {
            return;
        }

        var policy = new CacheItemPolicy();

        if (!cacheTime.HasValue)
        {

```

```

        cacheTime = _cacheTime;
    }

    policy.AbsoluteExpiration = DateTime.Now
        + TimeSpan.FromMinutes(cacheTime.Value);

    BinaryFormatter serializer = new BinaryFormatter();
    using (MemoryStream memStream = new MemoryStream())
    {
        serializer.Serialize(memStream, objectData);
        Cache.Add(new CacheItem(key, memStream.ToArray()), policy);
    }
}

/// Gets a value indicating whether the value associated
/// with the specified key is cached
public bool IsSet(string key)
{
    return Cache.Contains(key);
}

/// Removes the value with the specified key from the cache
public void Remove(string key)
{
    Cache.Remove(key);
}

public void RemoveByPattern(string pattern)
{
    foreach (var item in Cache)
    {
        if (item.Key.StartsWith(pattern))
        {
            Remove(item.Key);
        }
    }
}

/// Clear all cache data
public void Clear()
{
    foreach (var item in Cache)
    {
        Remove(item.Key);
    }
}
}

```

Etapă 2. Adăugați fiecărei metode din nivelul Service, posibilitatea de a prelua valoarea dorită din cache sau din baza de date.

Exemplu:

```

public Masina GetMasina(int id)
{
    string key = "masina_" + id;
    Masina masina;

    if (cacheManager.IsSet(key))

```

```

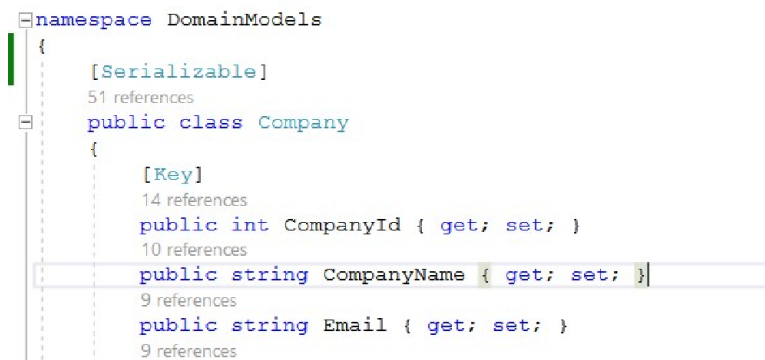
    {
        masina = cacheManager.Get<Masina>(key);
    }
    else
    {
        masina = stocareMasini.GetMasina(id);
        cacheManager.Set(key, masina);
    }

    return masina;
}

```

Observații:

1. Pentru a putea fi adăugate în cache, entitățile trebuie marcate cu atributul [Serializable].



```

namespace DomainModels
{
    [Serializable]
    51 references
    public class Company
    {
        [Key]
        14 references
        public int CompanyId { get; set; }
        10 references
        public string CompanyName { get; set; }
        9 references
        public string Email { get; set; }
        9 references
    }
}

```

Figura 1. Atribut [Serializable]

2. Dacă s-a integrat deja injectarea dependențelor,
 - a. trebuie injectat prin constructor și serviciul de cache (Figura 2.a)
 - b. trebuie înregistrat în containerul de dependențe serviciul de cache (Figura 2.b).



```

namespace BusinessLayer
{
    3 references
    public class CompaniesService: ICompanies
    {
        private ICache cacheManager;
        private CarStoreDbContext db;

        0 references
        public CompaniesService(ICarDbContext db, ICache cacheManager)
        {
            this.db = (CarStoreDbContext)db;
            this.cacheManager = cacheManager;
        }
    }
}

```

Figura 2.a. Injectare prin constructor a serviciului de cache

```

public class ContainerConfigurer
{
    1 reference
    public static void ConfigureContainer()
    {
        var builder = new ContainerBuilder();

        // Register your MVC controllers. (MvcApplication is the name of
        // the class in Global.asax.)
        builder.RegisterControllers(typeof(MvcApplication).Assembly);

        // Register individual types
        builder.RegisterType<MemoryCacheService>()
            .As<ICache>();

        builder.RegisterType<CarStoreDbContext>()
            .As<ICarDbContext>()
            .SingleInstance();
    }
}

```

Figura 2.b. Înregistrare în containerul de dependențe a serviciului de cache

Bibliografie

1. <https://www.c-sharpcorner.com/article/memory-cache-in-c-sharp/>